

Código fuente — PC4 Desarrollo de Software

Repositorio: <https://github.com/Melissa1221/pc4-desarrollo-software>

```
pom.xml
src/main/java/com/uni/petfinder/alertas/ActiveState.java
src/main/java/com/uni/petfinder/alertas/AlertCenter.java
src/main/java/com/uni/petfinder/alertas/AlertObserver.java
src/main/java/com/uni/petfinder/alertas/ClosedState.java
src/main/java/com/uni/petfinder/alertas/FoundState.java
src/main/java/com/uni/petfinder/alertas/LostPet.java
src/main/java/com/uni/petfinder/alertas/ReportState.java
src/main/java/com/uni/petfinder/alertas/Sighting.java
src/main/java/com/uni/petfinder/App.java
src/main/java/com/uni/petfinder/busqueda/AdoptionSearch.java
src/main/java/com/uni/petfinder/busqueda/ImageQuery.java
src/main/java/com/uni/petfinder/busqueda/LostCheckSearch.java
src/main/java/com/uni/petfinder/busqueda/SaleSearch.java
src/main/java/com/uni/petfinder/busqueda/SearchStrategy.java
src/main/java/com/uni/petfinder/busqueda/SearchTemplate.java
src/main/java/com/uni/petfinder/cuidadores/AlertCommand.java
src/main/java/com/uni/petfinder/cuidadores/AlertSwitch.java
src/main/java/com/uni/petfinder/cuidadores/CaregiverProfile.java
src/main/java/com/uni/petfinder/cuidadores/DisableAlertsCommand.java
src/main/java/com/uni/petfinder/cuidadores/EnableAlertsCommand.java
src/main/java/com/uni/petfinder/cuidadores/IdValidator.java
src/main/java/com/uni/petfinder/cuidadores/NameValidator.java
src/main/java/com/uni/petfinder/cuidadores/ProfileValidator.java
```

./pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.uni</groupId>
  <artifactId>petfinder</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
          <mainClass>com.uni.petfinder.App</mainClass>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

```

        </configuration>
    </plugin>
</plugins>
</build>
</project>

```

src/main/java/com/uni/petfinder/alertas/ActiveState.java

```

package com.uni.petfinder.alertas;

public class ActiveState implements ReportState {

    @Override
    public String name() {
        return "ACTIVO";
    }

    @Override
    public ReportState next() {
        return new FoundState();
    }
}

```

src/main/java/com/uni/petfinder/alertas/AlertCenter.java

```

package com.uni.petfinder.alertas;

import java.util.ArrayList;
import java.util.List;

// Subject del patrón Observer: mantiene la lista de suscriptores y les avisa
// cuando se reporta una mascota perdida dentro de su radio (RF 1.4).
public class AlertCenter {

    private final List<Subscription> subs = new ArrayList<>();

    public void subscribe(AlertObserver obs, double lat, double lon) {
        subs.add(new Subscription(obs, lat, lon));
    }

    public void unsubscribe(AlertObserver obs) {
        subs.removeIf(s -> s.obs == obs);
    }

    public void reportLostPet(LostPet pet, double radiusKm) {
        for (Subscription s : subs) {
            if (distanceKm(pet.getLat(), pet.getLon(), s.lat, s.lon) <= radiusKm) {
                s.obs.onLostPet(pet);
            }
        }
    }

    private double distanceKm(double lat1, double lon1, double lat2, double lon2) {
        double dLat = (lat1 - lat2) * 111;
    }
}

```

```

        double dLon = (lon1 - lon2) * 111;
        return Math.sqrt(dLat * dLat + dLon * dLon);
    }

    private static class Subscription {
        final AlertObserver obs;
        final double lat;
        final double lon;

        Subscription(AlertObserver obs, double lat, double lon) {
            this.obs = obs;
            this.lat = lat;
            this.lon = lon;
        }
    }
}

```

src/main/java/com/uni/petfinder/alertas/AlertObserver.java

```

package com.uni.petfinder.alertas;

// Observer: quien quiera recibir alertas de mascotas perdidas.
public interface AlertObserver {
    void onLostPet(LostPet pet);
}

```

src/main/java/com/uni/petfinder/alertas/ClosedState.java

```

package com.uni.petfinder.alertas;

public class ClosedState implements ReportState {

    @Override
    public String name() {
        return "CERRADO";
    }

    @Override
    public ReportState next() {
        return this; // estado final
    }
}

```

src/main/java/com/uni/petfinder/alertas/FoundState.java

```

package com.uni.petfinder.alertas;

public class FoundState implements ReportState {

    @Override
    public String name() {
        return "ENCONTRADO";
    }
}

```

```

@Override
public ReportState next() {
    return new ClosedState();
}
}

```

src/main/java/com/uni/petfinder/alertas/LostPet.java

```

package com.uni.petfinder.alertas;

// RF 1.1: mascota reportada como perdida por su dueño.
public class LostPet {

    private final String name;
    private final String species;
    private final String breed;
    private final String photo;
    private final String description;
    private final String ownerId;
    private final double lat;
    private final double lon;

    // el reporte arranca activo y va cambiando de estado (patrón State).
    private ReportState state = new ActiveState();

    public LostPet(String name, String species, String breed, String photo,
                    String description, String ownerId, double lat, double lon) {
        this.name = name;
        this.species = species;
        this.breed = breed;
        this.photo = photo;
        this.description = description;
        this.ownerId = ownerId;
        this.lat = lat;
        this.lon = lon;
    }

    public String getState() {
        return state.name();
    }

    // avanza el reporte al siguiente estado.
    public void advance() {
        state = state.next();
    }

    public String getName() {
        return name;
    }

    public String getSpecies() {
        return species;
    }

    // RF 1.2: coordenadas del reporte.

```

```

    public double getLat() {
        return lat;
    }

    public double getLon() {
        return lon;
    }

    // RNF 1.2: el id del dueño no se expone al ciudadano que avista.
    public String getOwnerId() {
        return ownerId;
    }

    public String getDescription() {
        return description;
    }
}

```

src/main/java/com/uni/petfinder/alertas/ReportState.java

```

package com.uni.petfinder.alertas;

// State: comportamiento del reporte según su estado (activo, encontrado, cerrado).
public interface ReportState {
    String name();
    ReportState next(); // estado al que pasa cuando avanza el reporte
}

```

src/main/java/com/uni/petfinder/alertas/Sighting.java

```

package com.uni.petfinder.alertas;

// RF 1.3: avistamiento registrado por un ciudadano anónimo.
public class Sighting {

    private final String photo;
    private final double lat;
    private final double lon;

    public Sighting(String photo, double lat, double lon) {
        this.photo = photo;
        this.lat = lat;
        this.lon = lon;
    }

    public String getPhoto() {
        return photo;
    }

    public double getLat() {
        return lat;
    }

    public double getLon() {

```

```

        return lon;
    }
}

```

src/main/java/com/uni/petfinder/App.java

```

package com.uni.petfinder;

import com.uni.petfinder.alertas.AlertCenter;
import com.uni.petfinder.alertas.AlertObserver;
import com.uni.petfinder.alertas.LostPet;
import com.uni.petfinder.busqueda.AdoptionSearch;
import com.uni.petfinder.busqueda.ImageQuery;
import com.uni.petfinder.busqueda.LostCheckSearch;
import com.uni.petfinder.busqueda.SaleSearch;
import com.uni.petfinder.busqueda.SearchTemplate;
import com.uni.petfinder.cuidadores.AlertSwitch;
import com.uni.petfinder.cuidadores.CaregiverProfile;
import com.uni.petfinder.cuidadores.DisableAlertsCommand;
import com.uni.petfinder.cuidadores.IdValidator;
import com.uni.petfinder.cuidadores.NameValidator;
import com.uni.petfinder.cuidadores.ProfileValidator;

import java.util.List;

// Demo que ejercita los patrones de comportamiento de cada módulo.
public class App {

    public static void main(String[] args) {

        System.out.println("=== Módulo 1: alertas (Observer + State) ===");
        AlertCenter center = new AlertCenter();

        AlertObserver ana = pet ->
            System.out.println("Ana recibe alerta: " + pet.getName() + " perdido cerca");
        center.subscribe(ana, -12.021, -77.031);

        AlertObserver luis = pet ->
            System.out.println("Luis recibe alerta: " + pet.getName());
        center.subscribe(luis, -12.20, -76.90);

        LostPet firulais = new LostPet("Firulais", "perro", "criollo", "foto.jpg",
            "collar rojo", "owner-1", -12.02, -77.03);
        System.out.println("Estado del reporte: " + firulais.getState());
        center.reportLostPet(firulais, 1.0);

        // el reporte avanza de estado (State).
        firulais.advance();
        System.out.println("Nuevo estado: " + firulais.getState());

        System.out.println("\n=== Módulo 2: buscador por imagen (Strategy + Template Method) ===");
        ImageQuery query = new ImageQuery("mascota.png");
        List<SearchTemplate> searches = List.of(
            new AdoptionSearch(), new SaleSearch(), new LostCheckSearch());
        for (SearchTemplate s : searches) {

```

```

        System.out.println(s.getClass().getSimpleName() + ": " + s.run(query));
    }

    System.out.println("\n=== Módulo 3: cuidadores (Command + Chain of Responsibility) ===");
    CaregiverProfile maria = new CaregiverProfile("María", "especializado");
    maria.acceptSpecies("perro");
    maria.setIdVerified(true);

    // cadena de validación antes de habilitar el perfil.
    ProfileValidator chain = new NameValidator();
    chain.linkWith(new IdValidator());
    System.out.println("¿Perfil habilitado? " + chain.validate(maria));

    // interruptor de alertas con Command (RF 3.3).
    AlertSwitch aSwitch = new AlertSwitch();
    aSwitch.press(new DisableAlertsCommand(), maria);
    System.out.println("¿Recibe alertas? " + maria.receiveAlerts());
}
}

```

src/main/java/com/uni/petfinder/busqueda/AdoptionSearch.java

```

package com.uni.petfinder.busqueda;

import java.util.List;

// RF 2.3: adopción devuelve solo protectoras/ONGs. La búsqueda real no se
// implementa; se deja el punto donde iría el motor.
public class AdoptionSearch extends SearchTemplate {

    @Override
    protected List<String> search(ImageQuery query) {
        return List.of("(resultados del catálogo de protectoras/ONGs)");
    }
}

```

src/main/java/com/uni/petfinder/busqueda/ImageQuery.java

```

package com.uni.petfinder.busqueda;

// RF 2.1 / RNF 2.1: consulta que llega al buscador. Los metadatos van en
// un mapa (JSON plano) para que el motor sea intercambiable.
import java.util.HashMap;
import java.util.Map;

public class ImageQuery {

    private final String imageFile;
    private final Map<String, String> metadata = new HashMap<>();

    public ImageQuery(String imageFile) {
        this.imageFile = imageFile;
    }
}

```

```

    public String getImageFile() {
        return imageFile;
    }

    public void put(String key, String value) {
        metadata.put(key, value);
    }

    public String get(String key) {
        return metadata.get(key);
    }
}

```

src/main/java/com/uni/petfinder/busqueda/LostCheckSearch.java

```

package com.uni.petfinder.busqueda;

import java.util.List;

// RF 2.5: verificar pérdida contrasta con las alertas activas. Búsqueda no
// implementada; aquí iría el cruce con la base de alertas.
public class LostCheckSearch extends SearchTemplate {

    @Override
    protected List<String> search(ImageQuery query) {
        return List.of("(coincidencias contra las alertas activas)");
    }
}

```

src/main/java/com/uni/petfinder/busqueda/SaleSearch.java

```

package com.uni.petfinder.busqueda;

import java.util.List;

// RF 2.4: venta muestra solo criaderos certificados. Búsqueda no implementada.
public class SaleSearch extends SearchTemplate {

    @Override
    protected List<String> search(ImageQuery query) {
        return List.of("(resultados de criaderos comerciales certificados)");
    }
}

```

src/main/java/com/uni/petfinder/busqueda/SearchStrategy.java

```

package com.uni.petfinder.busqueda;

import java.util.List;

// Strategy: contrato común de una búsqueda por imagen. Cada intención es una
// estrategia distinta e intercambiable (RNF 2.1).
public interface SearchStrategy {

```



```
List<String> run(ImageQuery query);
}
```

src/main/java/com/uni/petfinder/busqueda/SearchTemplate.java

```
package com.uni.petfinder.busqueda;

import java.util.List;

// Template Method: fija el flujo de una búsqueda (validar -> buscar -> formatear)
// y deja que cada intención complete el paso concreto de búsqueda.
public abstract class SearchTemplate implements SearchStrategy {

    // método plantilla (a la vez implementa la estrategia): el orden no cambia.
    @Override
    public final List<String> run(ImageQuery query) {
        validate(query);
        List<String> results = search(query);
        return format(results);
    }

    private void validate(ImageQuery query) {
        if (query.getImageFile() == null) {
            throw new IllegalArgumentException("Falta la imagen");
        }
    }

    // paso que cada intención define a su manera.
    protected abstract List<String> search(ImageQuery query);

    protected List<String> format(List<String> results) {
        return results; // hook por defecto; una intención puede sobrescribirlo.
    }
}
```

src/main/java/com/uni/petfinder/cuidadores/AlertCommand.java

```
package com.uni.petfinder.cuidadores;

// Command: acción sobre un perfil de cuidador.
public interface AlertCommand {
    void execute(CaregiverProfile profile);
}
```

src/main/java/com/uni/petfinder/cuidadores/AlertSwitch.java

```
package com.uni.petfinder.cuidadores;

// Invoker del Command: el interruptor que dispara la acción sobre el perfil.
public class AlertSwitch {

    public void press(AlertCommand command, CaregiverProfile profile) {
        command.execute(profile);
    }
}
```

```
}  
}
```

src/main/java/com/uni/petfinder/cuidadores/CaregiverProfile.java

```
package com.uni.petfinder.cuidadores;  
  
import java.util.ArrayList;  
import java.util.List;  
  
// Perfil de un cuidador. RF 3.1 / RF 3.2: rol y restricciones de servicio.  
public class CaregiverProfile {  
  
    private final String name;  
    private final String role; // solidario, profesional, especializado  
    private final List<String> species = new ArrayList<>();  
    private boolean idVerified; // RNF 3.1  
    private boolean receivesAlerts = true; // RF 3.3  
  
    public CaregiverProfile(String name, String role) {  
        this.name = name;  
        this.role = role;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getRole() {  
        return role;  
    }  
  
    public void acceptSpecies(String s) {  
        species.add(s);  
    }  
  
    public List<String> getSpecies() {  
        return species;  
    }  
  
    public boolean isIdVerified() {  
        return idVerified;  
    }  
  
    public void setIdVerified(boolean v) {  
        this.idVerified = v;  
    }  
  
    public boolean receivesAlerts() {  
        return receivesAlerts;  
    }  
  
    public void setReceivesAlerts(boolean v) {  
        this.receivesAlerts = v;  
    }  
}
```

```
}
```

src/main/java/com/uni/petfinder/cuidadores/DisableAlertsCommand.java

```
package com.uni.petfinder.cuidadores;

// RF 3.3: desactiva la recepción de alertas.
public class DisableAlertsCommand implements AlertCommand {

    @Override
    public void execute(CaregiverProfile profile) {
        profile.setReceivesAlerts(false);
    }
}
```

src/main/java/com/uni/petfinder/cuidadores/EnableAlertsCommand.java

```
package com.uni.petfinder.cuidadores;

// RF 3.3: activa la recepción de alertas.
public class EnableAlertsCommand implements AlertCommand {

    @Override
    public void execute(CaregiverProfile profile) {
        profile.setReceivesAlerts(true);
    }
}
```

src/main/java/com/uni/petfinder/cuidadores/IdValidator.java

```
package com.uni.petfinder.cuidadores;

// RNF 3.1: no se habilita el perfil sin documento validado.
public class IdValidator extends ProfileValidator {

    @Override
    protected boolean check(CaregiverProfile profile) {
        return profile.isIdVerified();
    }
}
```

src/main/java/com/uni/petfinder/cuidadores/NameValidator.java

```
package com.uni.petfinder.cuidadores;

public class NameValidator extends ProfileValidator {

    @Override
    protected boolean check(CaregiverProfile profile) {
        return profile.getName() != null && !profile.getName().isBlank();
    }
}
```

src/main/java/com/uni/petfinder/cuidadores/ProfileValidator.java

```
package com.uni.petfinder.cuidadores;

// Chain of Responsibility: cada validador revisa una condición y, si pasa,
// deja seguir al siguiente. Se usa para habilitar el perfil (RNF 3.1).
public abstract class ProfileValidator {

    private ProfileValidator next;

    public ProfileValidator linkWith(ProfileValidator next) {
        this.next = next;
        return next;
    }

    public boolean validate(CaregiverProfile profile) {
        if (!check(profile)) {
            return false;
        }
        if (next == null) {
            return true;
        }
        return next.validate(profile);
    }

    protected abstract boolean check(CaregiverProfile profile);
}
```